Eötvös Loránd University

Faculty of Informatics

Erasmus Mundus Joint Master in
Intelligent Field Robotic Systems

Globally Consistent Mapping in Indoor and Outdoor Environments Using Hybrid LiDAR SLAM

Author:
Zewdie Habtie Sisay
MSc Intelligent Field Robotic Systems (IFRoS)

Internal supervisor:
Zoltán Istenes
Professor

External supervisor:
Mohammad Albaja
Associate Professor, SMART

Budapest, 2025

Acknowledgements

Contents

Chapter 1

Introduction

1.1 Motivation

In recent years, robotics has evolved with the goal to develop intelligent robots that can assist or replace human beings in performing tasks. In order for these robots to operate effectively and efficiently in diverse environment, a set of modules that define the robot autonomy such as path planning, localization, mapping, perception, and manipulation are required. These modules are interconnected to each other in a sense that one can be an input to the other. In similar manner to how we humans understand and interact with our environment, robots also need to have these modules tightly integrated.

Map is an important prerequisite that helps a robot determine the pose of objects within its environment, enabling it to operate and perform tasks such as navigation, obstacle avoidance, and path planning. Both mapping and localization modules are usually combined and referred to as Simultaneous Localization and Mapping (SLAM) module where the robot builds the map while continuously localizing itself inside it. The planning module allows the robot to plan a path from its current pose (position, and orientation) to a goal pose. An algorithm that assists the robot to follow the planned path, a controller, is also considerably essential entity. Using perception module, robot senses and interpret the environment. In same way we humans use our sensing organs to sense our surrounding, robot uses its onboard sensors such as camera, Light Detection and Ranging (LiDAR), Radio Detection and Ranging (RADAR), Inertial Measurement Unit (IMU), Global Positioning System (GPS) and other sensors. The manipulation module enables the robot to interact with objects in its environment by performing tasks such as

grasping, picking, placing, and etc.

Map being an important prerequisite and SLAM being the state-off-the-art solution to generating it, there are several techniques to solving SLAM in the literature using different or similar methods. The two main techniques are online SLAM and full SLAM. These techniques rely on different sensor modalities with LiDAR SLAM and Visual SLAM being two the most widely used approaches.

1.2 Objectives

The primary objectives of the thesis are:

- To develop robust LiDAR SLAM that maintains global consistency of map both for indoor and outdoor environments by integrating LiDAR Inertial odometry, graph SLAM, LiDAR bundle adjustment.
- To incorporate robust loop closure detection technique with outlier rejection.
- To evaluate the systems performance against state-of-the-art SLAM methods using multiple datasets, analyzing metrics such as trajectory accuracy, map consistency, computational efficiency.

1.3 Research Questions

- Is LiDAR odometry alone sufficient to generate accurate map of complex environments?
- What are the limitations of Graph SLAM in generating consistent map especially in revisited locations?
- How effective is it to integrate LiDAR Inertial Odometry, Graph SLAM,and LiDAR Bundle Adjustment for generating globally consistent map of environments?

1.4 Scope and Limitation

1.5 Thesis Structure

The rest of the thesis is organized as follows. Chapter 2 discusses the literature review focusing in background foundations and describing the most related topics; LiDAR inertial Odometry, Online SLAM, Full SLAM, and LiDAR Bundle Adjustment. Chapter 3 provides detailed description of the methods used to solve entire pipeline of the system. Chapter 4 presents the result obtained for different datasets. Chapter 5 presents the detailed discussion of the results obtained. In chapter 6 a conclusion is provided.

Chapter 2

Literature Review

2.1 Introduction

The purpose of this chapter is to provide background knowledge and review existing research related to LiDAR-Based SLAM, state estimation, and loop detection.

In mobile robotics, map is an important prerequisite that enables robots to operate and perform different tasks such as navigation, obstacle avoidance, path planning, etc. SLAM is the most popular method in the literature to generate maps. Among the different methods of SLAM, vision-based and LiDAR technologies are the most popular in the literature. Although each SLAM method has its advantages and drawbacks, they are essential for different tasks. LiDAR technologies will likely remain essential for years to come due to their numerous advantages: robust to various lighting conditions, accurate 3D measurements with higher Field of View (FoV), among others, despite their high cost and bulkiness to install in small-scale robots like drones. Solid-state LiDARs are cost-effective, lightweight, and precise, making them ideal for UAVs in mapping and complex environments.

Although these solid-state LiDARs have good potential, they have presented new challenges to SLAM solutions: 1) When the robot is operating in a cluttered environment, there not be strong features with geometric shapes like edges and planes. This causes the LiDAR SLAM to degenerate easily. 2) Fusing many feature points with IMU is challenging. 3) LiDAR points are sequentially sampled resulting in each point having a different timestamp which causes motion distortion that severely affects the point registration [1].

Several works exist in the literature on LiDAR SLAM. Some of these works use filter-based (Kalman filter) approach while others use optimization-based(factor graphs) for the state estimation. Although filter-based approaches are simple in terms of implementation and computational requirements, they accumulate errors over time causing a larger drift. Optimization-based approaches on the other hand formulate the SLAM problem as non-linear problem and optimize the whole robot trajectory and the map minimizing the drift, more specifically during loop closure. In the following sections, we introduce basic probabilistic formulations, algorithms, and concepts related to SLAM that are used in the research. We also review the most related and relevant works.

2.2 Introduction to joint probability

In robotics, sensor measurements, controls, and robot states are modeled as random variables. This modeling helps to correctly incorporate the uncertainties that exist due to the sensor measurements, the motion model of the robot, or the environment itself. These variables can take multiple values following probabilistic laws. Probabilistic inference calculates the probability distributions of derived random variables, such as sensor data. Let $\mathbf{X}$ denote a random variable and x a possible event. The probability of $\mathbf{X}$ taking a value x is denoted as

$$p(X = x) \tag{2.1}$$

Eq (2.1) can also be written as *p(x)* for simplicity. The sum of all possible probabilities of discrete random variable $\mathbf{X}$ sum to 1.

$$\sum_{x} p(x) = 1 \tag{2.2}$$

To make estimation in a continuous space, a set of continuous random variables with probability density functions(PDFs) are used. Most commonly a normal distribution with mean μ and variance σ^2, shown in Eq. (2.3), is used.

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) \tag{2.3}$$

The PDF for multivariate random variable, usually expressed as vector form, is shown in Eq. (2.4) where $\mu \in \mathbb{R}^n$ is the mean and Σ is the *covariance matrix*.

$$p(x) = \det(2\pi\Sigma)^{-\frac{1}{2}} \exp\left\{-\tfrac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right\} \tag{2.4}$$

To express the joint probability distribution of two random variables $\mathbf{X}$ and $\mathbf{Y}$, we use the notation $p(x,\ y) = p(\mathbf{X} = x \text{ and } \mathbf{Y} = y)$. If $\mathbf{X}$ and $\mathbf{Y}$ are independent, $p(x,y) = p(x)\ p(y)$.

The conditional probability of x given y is denoted as:

$$p(x \mid y) = \frac{p(x, y)}{p(y)} \tag{2.5}$$

Leveraging the conditional probability shown in Eq. (2.5), Bayes rule can be derived and put as in the following Eq. (2.6).

$$p(x \mid y) = \frac{p(y \mid x)\ p(x)}{p(y)} \tag{2.6}$$

It is also possible to condition the Bayes rule described in Eq. (2.6) on arbitrary other variables, z, as shown in Eq. (2.7**)**.

$$p(x \mid y, z) = \frac{p(y \mid x, z)\ p(x \mid z)}{p(y \mid z)} \tag{2.7}$$

Similarly, probabilities of independent random variables x and y can be combined on other variable z as in Eq. (2.8).

$$p(x, y \mid z) = p(y \mid z)\ p(x \mid z) \tag{2.8}$$

2.3 LiDAR Odometry and LiDAR Inertial Odometry

Accurate 3D mapping with LiDARs requires efficiently registering the point clouds to a global map frame. This efficiency of registration depends on accurate estimation of the LiDAr pose in continuous motion. Independent motion estimation systems such as Global Navigation Satellite Systems (GNSS) or Inertial Navigation Systems(INS) can be used for registration,however GNSS systems do not provide continuous signal in diverse environments and INS accumulate drifts over time resulting in inconsistency in the 3D map.

LiDAR odometry (LO) is a continuous estimation of the motion of the vehicle by analyzing successive 3D point clouds captured by LiDAR sensor mounted on it. This works by tracking the changes in the structure of the environment that is achieved aligning the current scan to the previous and computing the relative motion. LO methods are robust

in environments with rich features (planes, and edges). The most widely used registration method Iterative Closest Point (ICP) is one example that can be used to estimate motion with LO.

However, LO suffers from drift and degeneration specially in challenging environments such as repetitive structures and dynamic objects. Fusing high frequency IMU data with LiDAR can rescue LO and by reducing the drift thereby improving the motion estimation. This fusion of LiDAR and IMU for motion esstimation is known as LiDAR Inertial Odometry (LIO). Several researches [2, 3, 1] have been conducted demonstrating the effectiveness LIO for motion estimation and 3D mapping.

2.4 Lie Theory and 3D pose representation

The proper formulation of many estimation algorithms in robotics demands stability, precision, and consistency of the solutions. To achieve this, proper modeling of states and measurements, relationship between them, and uncertainties of the relation is important [4]. To this end, a smooth topological surfaces of Lie groups known as manifolds are designed. Manifolds are mathematical curved spaces that resemble euclidean space ($\mathbb{R}^n$) but they actually have more complicated global structure.

2.4.1 Lie groups

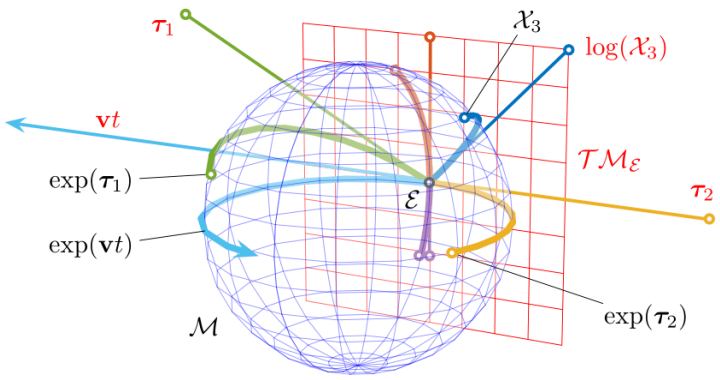


Figure 2.1: Relationship between Lie Groups and Lie algebras

A Lie group is a group that is also a smooth manifold, with group operations (multiplication and inversion) that are differentiable. In robotics, Lie groups model

continuous transformation groups such as rotations and rigid body motions. They enable representing and composing robot poses in a mathematically consistent way.

Orthogonal Groups and Special Orthogonal Groups

The orthogonal group O(n)represents all nxn matrices that preserve length and angles. The special orthogonal group, SO(n) consists of a set of all rotation matrices with determinant of +1. To represent 3D orientation of links, rotating sensors, to compute jacobian of motion models in robotics an SO(3) is used. Eq. (2.9) shows the orthogonal groups where $GL(n)$ is a general linear group of nxn matrices with non-zero determinant.

$$\begin{aligned} O(n) &= \left\{\mathbf{R} \in GL(n) : \mathbf{R}^\top \mathbf{R} = \mathbf{I}_{n\times n}\right\} \\ SO(n) &= \{\mathbf{R} \in O(n) : \det(\mathbf{R}) = 1\} \end{aligned} \tag{2.9}$$

Euclidean Groups and Special Euclidean Groups

The euclidean groups, E(n) and special euclidean groups, SE(n), the rigid transformation(rotation, and translation) in n dimensional space, see Eq. (2.10).

$$\begin{aligned} E(n) &= \left\{ \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} : R \in O(n),\, t \in \mathbb{R}^n \right\} \subset GL(n+1) \\ SE(n) &= \left\{ \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} : R \in SO(n),\, t \in \mathbb{R}^n \right\} \subset GL(n+1) \end{aligned} \tag{2.10}$$

2.4.2 Lie algebra

The Lie algebra $\mathbf{T_E}\mathcal{M}$ (red plane) to the Lie group's manifold $\mathcal{M}$ (blue sphere) is a tangent space at the identity denoted as $\mathbf{E}$ in figure 2.1. Each element in $\mathbf{T_E}\mathcal{M}$ has its equivalent on $\mathcal{M}$. For example, $\mathbf{v}$t produces path $exp(\mathbf{v}t)$, and log(X) corresponds to $\mathbf{X}$ in figure 2.1.

The group $SO(3)$ is a Lie group, while the space $\mathfrak{so}(3)$ is its corresponding Lie algebra. The latter is the tangent space at the identity of $SO(3)$. The Lie group is a complicated nonlinear object, while its Lie algebra is a vector space that is simpler to work with.

$$\begin{aligned} exp &: g \longrightarrow G \\ log &: G \longrightarrow g \end{aligned} \tag{2.11}$$

The operators $\boxplus$ and $\boxminus$ can express addition and subtraction for $\mathbf{X}$, $\mathbf{Y} \in \mathcal{M}$ and $\hat{\mathbf{u}} \in \mathbf{T_E}\mathcal{M}$.

$$\begin{aligned} \mathbf{X} \boxplus \mathbf{u} &\doteq \mathrm{Exp}(\mathbf{u}) \circ \mathbf{X}, \\ \mathbf{X} \boxminus \mathbf{Y} &\doteq \mathrm{Log}(\mathbf{X} \circ \mathbf{Y}^{-1}). \end{aligned} \tag{2.12}$$

2.5 Introduction to Bundle Adjustment

Bundle adjustment (BA) is a technique used to jointly solve both the 3D structure and camera poses in many visual applications such as structure from motion (SfM) and Visual-SLAM (V-SLAM). It works to minimize the re-projection error, see figure 2.2, between the observed 2D features in the images and projected 3D points(features), changing both the camera poses and the pose of 3D points to minimize the error.

One important thing to consider when working with BA is its cost. The computational complexity of BA is $\mathbf{O}(qN+lM)^3$ where N is the number points, M is the number of poses, q and l the number parameters for the points and the camera respectively. To mitigate this drawback of BA, different ways can be incorporated including using windowed BA with small window size, and optimizing only on poses and keeping 3D feature points fixed[**bibid**] paraphrase this paragraph ADD BA EQUATIONS

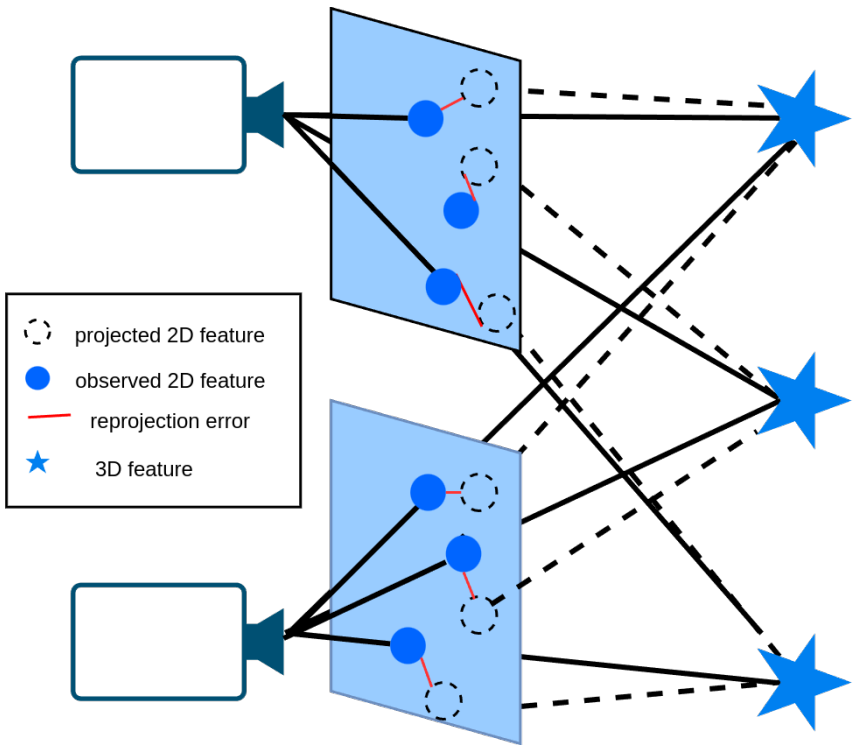


Figure 2.2: Visual BA

2.5.1 LiDAR Bundle Adjustment

In large-scale SLAM applications, such as at the city scale, map inconsistencies are common, even after Pose Graph Optimization (PGO), since PGO optimizes only relative pose errors between two LiDAR frames. Additionally, loop closure errors can significantly impact the global consistency of the map [5]. To address this, LiDAR BA (LBA) can be employed to improve global consistency. Furthermore, vertical misalignments, which most LIO systems are vulnerable for, can be corrected using LBA, although GPS data, if available, may also help mitigate vertical drift.

Similar to visual BA, the LBA problem can be formulated to jointly determine the LiDAR poses and the global 3D point cloud map thereby decreasing the drift caused by accumulation of scan registration errors. Even though LBA seems simpler than visual BA due to accurate depth measurements, its formulation is not trivial. Due to the sparse and non repetitive nature of LiDAR point clouds, the natural formulation used in visual BA to minimize the re-projection error does not apply to LBA [6].

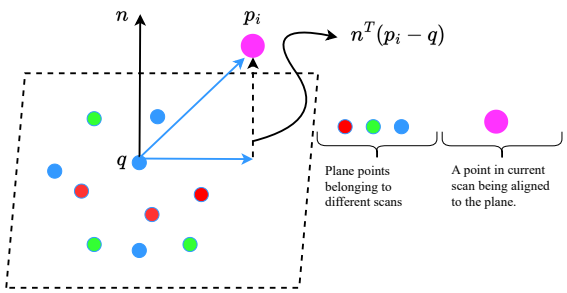


Figure 2.3: A plane in space and corresponding feature points drawn from different scans

A work by Liu et al. [7] introduced a formulation of BA on sparse LiDAR feature points with edges and planes to refine map incrementally built with LiDAR SLAM. In this formulation, unlike visual BA, the plane or edge parameters (the normal vector $\mathbf{n}$, and point in the plane $\mathbf{q}$, Eq. 2.13) are first analytically solved such that the BA problem is only related to LiDAR poses and such parameters are eliminated in each iteration of optimization [8]. This drastically reduces the cost of BA by enforcing it to only perform the so called motion-only BA without the need to optimize the 3D points.

$$\min_{\mathbf{n},\mathbf{q}} \frac{1}{N} \sum_{i=1}^{N} \left(\mathbf{n}^T (\mathbf{p}_i - \mathbf{q}) \right)^2 \tag{2.13}$$

Eq. (2.14) shows the direct formulation of LBA that minimizes the sum of squared distances of each LiDAR point to the plane where $\mathbf{p}_i$ is a point observed from pose $\mathbf{T}_i$, and $\mathbf{A}$ is the covariance matrix of the observed points, λ_k is the k-th smallest eigenvalue of $\mathbf{A}$, $\mathbf{u}_k$ is the corresponding eigenvector.

$$\begin{aligned}(\mathbf{T}^*, \mathbf{n}^*, \mathbf{q}^*) &= \arg\min_{\mathbf{T},\mathbf{n},\mathbf{q}} \frac{1}{N}\sum_{i=1}^{N}\Big(\mathbf{n}^T(\mathbf{p}_i - \mathbf{q})\Big)^2 \\ &= \arg\min_{\mathbf{T}}\left(\min_{\mathbf{n},\mathbf{q}} \frac{1}{N}\sum_{i=1}^{N}\Big(\mathbf{n}^T(\mathbf{p}_i - \mathbf{q})\Big)^2\right) \\ &= \lambda_3(\mathbf{A}), \quad \text{if } \mathbf{n}^* = \mathbf{u}_3,\ \mathbf{q}^* = \bar{\mathbf{p}} \\ \text{where} \quad \bar{\mathbf{p}} &= \frac{1}{N}\sum_{i=1}^{N}\mathbf{p}_i, \quad \mathbf{A} = \frac{1}{N}\sum_{i=1}^{N}(\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T \end{aligned} \tag{2.14}$$

The optimization on the scan poses $\mathbf{T}$ finally corresponds to minimizing the eigenvalues of matrix $\mathbf{A}$ in Eq. (2.14). The formulation defined in Eq. (2.14) requires the set of points, $\mathbf{p}_i$ acquired from set of scan poses $\mathbf{T}$, belonging to the same plane feature to be defined. Adaptive voxelization technique [7] is used to define the points that belong to the plane. If all points in the voxel lie on the plane by examining the eigenvalue of the point covariance matrix, that voxel is kept in memory otherwise the voxel is further subdivided into eight octants until a certain minimum threshold voxel size is reached. This generates voxel map with voxels of different sizes adapted to the environment.

The voxilization process described above extends to non-planar features such as curved surface when the voxel map is performed at a finer level. The maximum depth of a tree and the minimum number of points contained in a voxel are the two conditions used to stop the recursive subdivision of a voxel.

2.6 Overview and types of SLAM

There are several types of SLAM solutions. Depending on the type of exteroceptive sensors used, LiDAR SLAM and V-SLAM are the most commonly used types. LiDAR SLAM uses laser sensors to measure distances and create precise 3D maps, making it highly effective in dark or low-feature environments. However, its high cost and power consumption make it less suitable for low-cost robotics. On the other hand, V-SLAM depends on monocular, stereo, RGB-D or event based cameras to detect visual features and track motion, providing a more affordable and lightweight solution. While V-SLAM captures

detailed textures and colors, it struggles in low-light conditions and areas with few visual features. Despite these differences, both types are essential in robotics and autonomous systems. To improve accuracy, many modern applications combine both techniques using sensor fusion, taking advantage of their strengths to enhance localization and mapping performance. Based on the solution methodology used, SLAM can be further categorized into online SLAM and Full SLAM.

2.6.1 Online SLAM

Online SLAM methods estimate the current state x_t and the whole map m based on all past observations $z_{1:t}$ and controls $u_{1:t}$. These methods use Kalman Filter(KF), Extended Kalman Filter (EKF), Particle Filter (PF), and Unscented Kalman Filter (UKF)[9]. EKF-SLAM is one of the most popular online SLAM algorithms that uses Bayes filter to estimate the robot poses and landmark locations, assuming Gaussian distributions and local linearity. Bayes filter is a recursive filter for state estimation that includes two steps, prediction and update as shown in Eq (2.15), where $\bar{belief}(x_t)$ represents the prior belief about the robot state at time t before incorporating the latest observation (prediction step) and $belief(x_t)$ is a posterior belief that defines the robots refined state after incorporating latest observation (update step).

$$\begin{aligned} \bar{belief}(x_t) &= \int p(x_t|x_{t-1}, u_t)\, belief(x_{t-1})\, dx_{t-1}, \\ belief(x_t) &= p(z_t|x_t)\bar{belief}(x_t). \end{aligned} \tag{2.15}$$

Map generated with online SLAM is not accurate due to the linearization errors accumulated over time, local state estimation and absence of loop closure constraint to refine accumulated errors.

2.6.2 Full SLAM

FuLL SLAM aims to solve the problem of finding the most probable sequence of robot poses $x_{1:t}$ and the map m, given all sensor measurements $z_{1:t}$ and control inputs $u_{1:t}$[10]. It is mostly used to build consistent map using point clouds and solve SfM problems, specially to handle loop closure events.

Bayesian network

One approach to solving the Full SLAM problem is using Bayesian network where the problem is formulated as joint distribution of all the variables($x_{1:t}, z_{1:t}, u_{1:t}$, and m). It is a probabilistic directed graph model for solving full SLAM where the entire problem is formulated as joint distribution of the robot poses ($x_{1:M}$), the measurements ($z_{1:K}$), controls ($u_{1:M}$), and landmarks ($l_{1:N}$). The joint probability distribution of the whole SLAM is defined as in Eq. (2.16) where the variables X, L, U, and Z are all robot poses, all landmarks, all control inputs, and all measurements respectively Eq.(2.17). Figure 2.4 shows a typical Bayes network where every node encodes a conditional probability on the variable and its parent nodes.

$$P(X, L, U, Z) \propto P(\mathbf{x}_0) \prod_{i=1}^{M} P(\mathbf{x}_i \mid \mathbf{x}_{i-1}, \mathbf{u}_i) \prod_{k=1}^{K} P(\mathbf{z}_k \mid \mathbf{x}_{i_k}, \mathbf{l}j_k) \tag{2.16}$$

$$\begin{aligned} X &= \{\mathbf{x}_i\}, \quad i \in 0 \dots M \\ L &= \{\mathbf{l}_j\}, \quad j \in 1 \dots N \\ U &= \{\mathbf{u}_i\}, \quad i \in 1 \dots M \\ Z &= \{\mathbf{z}_k\}, \quad k \in 1 \dots K \end{aligned} \tag{2.17}$$

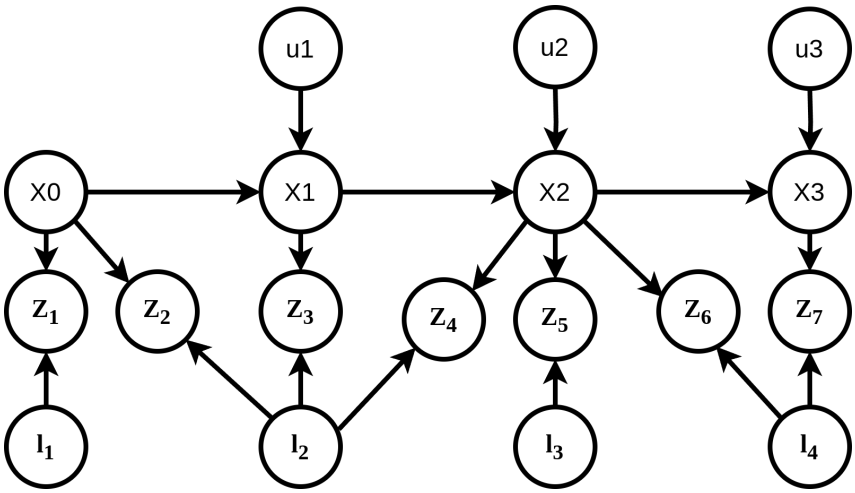


Figure 2.4: Probabilistic Bayesian net SLAM model

Factor Graph

Bayesian network uses only arrows to define dependency between variables which does not encode much of information. Effortlessly, Bayesian network can be converted to factor graphs by conditioning on sensor data [11]. Figure 2.5 shows the factor graph obtained by converting the Bayesian network shown in figure 2.4. Each node in the Bayesian network

is split in both variable node and factor node in the graph. The factor is connected to a variable node and parent variable nodes in the graph.

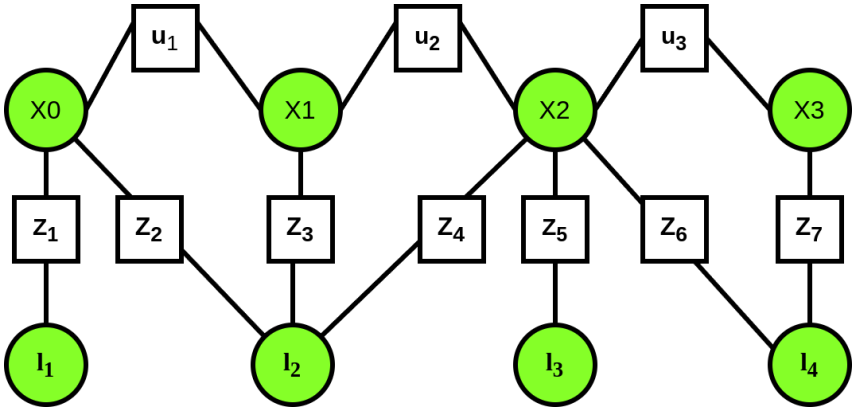


Figure 2.5: Full SLAM model with factor graph

Factor Graphs represent the probability distribution as a bipartite graph with variable nodes (green circles in Fig. 2.5) and factor nodes (white squares in Fig. 2.5) where edges can only connect variable nodes via factor nodes. These factor nodes encode information that enters into the system and the graph captures the propagation of these information to the hidden states that we intend ot estimate. We denote variable nodes as $\mathbf{x}$ and factor nodes as $\boldsymbol{\phi}$. The factors define the probabilistic constraint motion model ($\boldsymbol{\phi}_i$) and the likelihoods measurements ($\boldsymbol{\phi}_\mathbf{k}$). The joint probability distribution of the problem is then computed as the product of all factors in the graph Eq. (2.18). This graph structure allows efficient optimization via methods such as Gaussian Belief Propagation or Nonlinear Least Squares Optimization. Factor graph is the core of the prevalent SLAM technique known as pose graph optimization (PGO). Most of the LiDAR SLAM works [8, 3, 12], even the most recent KISS(Keep It Small and Simple) SLAM [13] use PGO with factor graph to generate 3D map of the environment from LiDAR point clouds.

$$p(\mathbf{X} \mid \mathbf{Z}) \propto \prod_{k=1}^{K} \phi_k \quad \text{where } \phi_k \geq 0 \quad \forall K \tag{2.18}$$

Solving the SLAM problem with factor graph is exactly determining the values of the unknown variable nodes($\mathbf{X}$) that maximize the agreement with the uncertain measurements($\mathbf{Z}$). This is known as maximum a posteriori estimate (MAP). Solving for the states (variable nodes) that maximize the posterior $p(X \mid Z)$ Eq. (2.19) can

be performed by exploiting non-linear optimization methods.

$$X^{MAP} = \arg\max_{X} p(X|Z) = \arg\max_{X} \frac{p(Z|X)p(X)}{p(Z)} \tag{2.19}$$

2.7 Related works

Ji Zhang et al. [14] proposed a real-time method for LiDAR odometry and mapping (LOAM). They achieved low-drift results without the need to use IMU and high-accuracy range measurements. In their method, the costly SLAM problem is divided into to algorithms where one algorithm computes the odometry at a higher frequency and the other algorithm runs at a much lower frequency to register point cloud. While it achieves impressive accuracy and real-time performance, it lucks inertial fusion and loop closures are not considered.

A method by Xu et al. [1] presented a novel approach to LIO called *Fast, Robust LiDAR Inertial Odometry package by tightly-coupled Iteratied Kalman Filter* (FAST-LIO) by proposing a new formula for computing the Kalman gain, which demonstrates equivalence to the conventional formula but reduces computation complexity based on state dimensions rather than measurement dimensions. An Iterative Kalman Filter(IKF) is used to minimize the point-to-plane distance during the registration of scan to map. The packages operates effectively on a small-scale quadrotor with fast motion resulting in real-time performance and stable odometry output and mapping at a frequency of $50HZ$. Experiments conducted in varied environments validate the system's resilience against fast motion and vibration noise. Despite its resilience to fast motion and cluttered environments, FAST-LIO does not include loop closure or global pose graph optimization causing accumulation of drift over long trajectories. Hence, in our research, we intended to include loop closure constraint on top of FAST-LIO to correct accumulated drifts over time. To this end, we conducted a review of available loop detection methods and present the relevant and robust method.

Gupta et al.[15] proposed a robust loop closure detection pipeline for LiDAR-based SLAM that effectively handles variations in LiDAR sensors, including different scanning patterns, fields of view, and resolutions. Their approach generates local maps from consecutive LiDAR scans and employs a ground alignment module to ensure consistency across

diverse motion profiles. A key feature of their method is the use of density-preserving Bird's Eye View (BEV) projections, from which ORB feature descriptors are extracted for efficient place recognition. These descriptors are stored in a binary search tree, enabling fast retrieval while mitigating perceptual aliasing through self-similarity pruning.

Kim et al. [16] presented Scan Context, a novel spatial descriptor designed for outdoor place recognition using 3D LiDAR scans. Unlike traditional histogram-based methods [17, 18, 19], Scan Context encodes the entire point cloud into a matrix format, preserving the absolute geometrical structure of the environment. This approach utilizes a cosine distance measure for similarity scoring and implements a two-phase search algorithm to enhance loop detection efficiency, particularly in urban settings. Experimental results demonstrate that Scan Context significantly outperforms existing global descriptors in various datasets, providing robust performance against noise and viewpoint changes.

A work by Xiyuan Liu et al. [8] presented a novel approach for large-scale LiDAR mapping by introducing a hierarchical LiDAR bundle adjustment (HBA) framework. The method aims to improve the consistency and accuracy of LiDAR-based maps by formulating a global optimization problem that jointly refines multiple scans. Unlike conventional SLAM techniques that rely on pairwise constraints, HBA leverages a hierarchical structure to balance local precision and global consistency, mitigating drift accumulation over extended trajectories. The authors demonstrate the effectiveness of their approach through extensive experiments on real-world datasets, showcasing its advantages in handling large-scale environments with minimal loop closures. Their results indicate that HBA outperforms existing methods in terms of mapping accuracy and robustness, particularly in scenarios with sparse revisits. A key contribution of this work is the introduction of an efficient optimization strategy that scales well with the number of LiDAR scans. By organizing the LiDAR data into hierarchical layers, the proposed framework enables efficient refinement without requiring excessive computational resources. The authors also incorporate a robust outlier rejection mechanism to enhance data association, ensuring reliable pose estimation in challenging conditions. Furthermore, the method integrates seamlessly with existing LiDAR-based odometry pipelines, making it adaptable to various robotic platforms. However, HBA map consistency degrades if there are more revisits and loop closures especially if the divergence in the map due to drift is larger than the maximum voxel size, BA might have slow convergence rate.

Wei Xu et al. [2] significantly improved upon the original work [1] by introducing two key advancements: direct point registration and the incremental k-D tree (*ikd-Tree*). Unlike [1], which relies on hand-crafted feature extraction, the new approach registers raw LiDAR points directly to the map, increasing accuracy and adaptability to different LiDAR sensors. The newly introduced ikdTree enables efficient incremental updates, dynamic re-balancing, and on-tree downsampling, reducing computational load while maintaining real-time performance. These improvements allow a new approach to achieve higher accuracy at a lower computational cost, making it more robust for diverse environments, including those with small FoV LiDARs and aggressive motion. Compared to their original work, the overall processing time for a single scan was improved by approximately a factor of ten.

Chapter 3

Methodology

3.1 System Overview

The proposed system integrates FAST-LIO2 [2] for LiDAR-inertial odometry, Scan Context for loop closure detection, PGO using GTSAM for correcting drifts in the odometry, and finally HBA for global refinement of the map. The diagram shown in figure 4.1 illustrates the integrated components used in system pipeline where the LIO and PGO can run in real-time but due to the computational and memory cost, HBA is run offline.

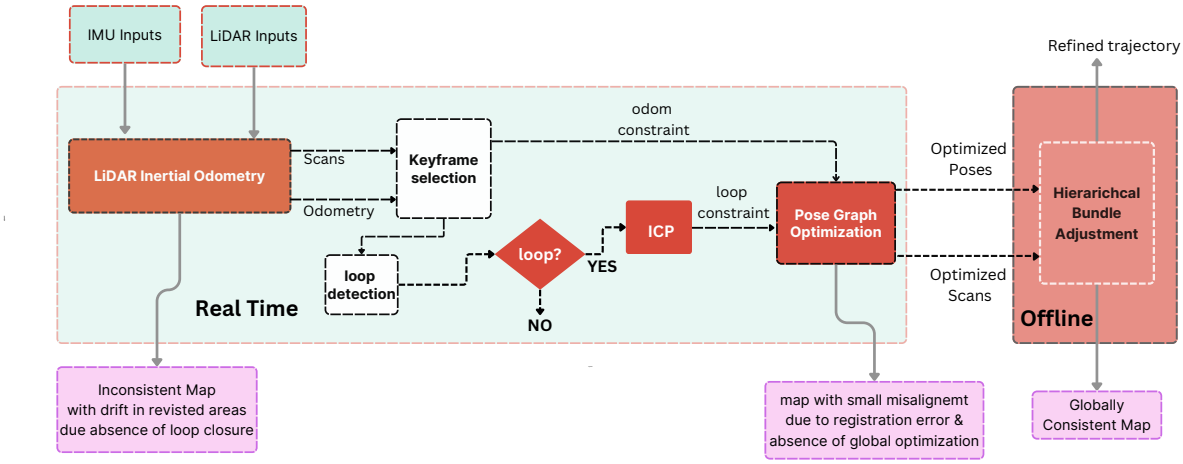


Figure 3.1: Block diagram of the overall system pipeline.

Algorithm 1 LiDAR-Inertial Odometry with Global Optimization

Input: $\mathcal{I}$ (IMU data), $\mathcal{L}$ (LiDAR scans)
Output: $\mathcal{M}^*$ (optimized map), $\mathcal{T}^*$ (refined trajectory)
Initialize: $\mathcal{K}$(Keyframe set) $\leftarrow \emptyset$, $\mathcal{G}$(Bipartite graph) $\leftarrow \emptyset$
while new $(\mathcal{I}_t, \mathcal{L}_t)$ available **do**
    $\hat{\mathbf{T}}_t \leftarrow \text{LIO}(\mathcal{I}_t, \mathcal{L}_t)$ 👉 FAST-LIO2
    if IsKeyframe$(\hat{\mathbf{T}}_t)$ **then**
        $\mathcal{K} \leftarrow \mathcal{K} \cup \{k_t\}$ 👉 Add keyframe
        $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathcal{C}_{\text{odom}}(k_t)\}$ 👉 Add odometry constraint
        if DetectLoop$(k_t, \mathcal{K})$ **then**
            $\mathcal{G} \leftarrow \mathcal{G} \cup \{\mathcal{C}_{\text{loop}}(k_t, k_l)\}$ 👉 Add loop closure constraint
            $\{\mathbf{T}_i^*\} \leftarrow$ OptimizePoseGraph$(\mathcal{G})$
        end if
    end if
end while
$\{\mathbf{T}_i^{**}, \mathcal{M}^*\} \leftarrow \text{HBA}(\{\mathbf{T}_i^*\}, \mathcal{K})$
return $\mathcal{M}^*$, $\mathcal{T}^* = \{\mathbf{T}_i^{**}\}$

FAST-LIO2 outputs odometry and updated LiDAR scan frames at a frequency of 10-100Hz. The scan frames are updated according to the state estimation module that includes prediction and update steps. Aggregating these scans using the odometry poses can generate accurate map for short trajectories because LIO performs well for sequential registration of scans. However after long trajectory and revisiting the same locations the map diverges due to accumulated drifts in the odometry.

In the subsequent sections, detailed description of each of the components used in the system will be presented.

3.2 Description of FAST-LIO2

To output odometry at higher frequency, a state estimation module fuses both IMU and LiDAR inputs with an IKF. This involves prediction of the state based on IMU input only also called forward propagation, then iteratively update it with LiDAR inputs until the residual drops below a certain threshold. The state vector $\mathbf{x}$ is a 24-dimensional vector containing the state variables shown in Eq. (3.1).

$$\begin{aligned} \mathbf{x} &\triangleq ({}^G\mathbf{R}_I^T, {}^G\mathbf{P}_I^T, {}^G\mathbf{V}_I^T, \mathbf{b}_\omega^T, \mathbf{b}_a^T, {}^G\mathbf{g}^T, {}^I\mathbf{R}_L^T, {}^I\mathbf{P}_L^T)^T \in \mathcal{M} \\ \mathcal{M} &\triangleq SO(3) \times \mathbb{R}^{15} \times SO(3) \times \mathbb{R}^3, \quad \dim(\mathcal{M}) = 24 \\ \mathbf{u} &\triangleq \begin{bmatrix} \boldsymbol{\omega}_m^T & \mathbf{a}_m^T \end{bmatrix}^T, \quad \mathbf{w} \triangleq \begin{bmatrix} \mathbf{n}_\omega^T & \mathbf{n}_a^T & \mathbf{n}_{b\omega}^T & \mathbf{n}_{ba}^T \end{bmatrix}^T \end{aligned} \tag{3.1}$$

${}^{G}\mathbf{R}_I^T$, ${}^{G}\mathbf{P}_I^T$, and ${}^{G}\mathbf{V}_I^T$ denote orientation, position, the linear velocity of IMU in the global frame respectively. The $\mathbf{b}_\omega^T$ is the bias in angular velocity, $\mathbf{b}_a^T$ is bias in acceleration of IMU, and ${}^{G}\mathbf{g}^T$ is gravity. ${}^{I}\mathbf{R}_L^T$, and${}^{I}\mathbf{P}_L^T$ denote the extrinsic calibration between IMU and LiDAR sensors. The vector $\mathbf{u}$ represents IMU inputs, and $\mathbf{w}$ denotes the noise vector.

3.2.1 Prediction Step

The continuous model is discretized at the IMU sampling period $\triangle t$ and the state is estimated using the forward propagation as shown in Eq. (3.2) where the function $\mathbf{f}$ is the state transition model as described in [2]. In the forward propagation, prediction step runs at frequency of $\frac{1}{\Delta t}$ until a (next) LiDAR scan comes in. The state and covariance matrix are propagated with the process noise $\mathbf{w}$ set to zero as in Eq. (3.3) where Q_i is the noise covariance, $\mathbf{F}_{\bar{\mathbf{x}}_i}$ and $\mathbf{F}_{\mathbf{w}_i}$ are jacobians of $\mathbf{f}$ with respect to state and noise respectively.

$$x_{i+1} = x_i \boxplus (\Delta t f(x_i, u_i, w_i)) \tag{3.2}$$

$$\begin{aligned} \hat{\mathbf{x}}_{i+1} &= \hat{\mathbf{x}}_i \boxplus \Big(\Delta t \mathbf{f}(\hat{\mathbf{x}}_i, \mathbf{u}_i, \mathbf{0})\Big); \quad \hat{\mathbf{x}}_0 = \bar{\mathbf{x}}_{k-1}, \\ \hat{\mathbf{P}}_{i+1} &= \mathbf{F}_{\bar{\mathbf{x}}_i} \hat{\mathbf{P}}_i \mathbf{F}_{\bar{\mathbf{x}}_i}^T + \mathbf{F}_{\mathbf{w}_i} \mathbf{Q}_i \mathbf{F}_{\mathbf{w}_i}^T; \quad \hat{\mathbf{P}}_0 = \bar{\mathbf{P}}_{k-1}, \end{aligned} \tag{3.3}$$

3.2.2 Update Step

The update step corrects the predicted state by minimizing point-to-plane residuals between LiDAR points and the map. The residual z_i^k of each LiDAR point ${}^{L}\mathbf{p}_i$ is computed as its distance to a nearest local plane in the map. This residual Eq. (3.4) quantifies the alignment error at the kth iteration.

Since the observation contains large number points($\approx$ 100K points), inverting a matrix in the dimension of observation is costly. Hence a new formulation Eq. (3.5) of the kalman gain $\mathbf{K}$ that needs to invert a matrix in the dimension of state is derived. Using the residual $\mathbf{z}_k^\kappa$ and the $\mathbf{K}$, the predicted state is updated as shown in Eq. (3.6).

$$\mathbf{z}_j^\kappa = \mathbf{u}_j^T \left({}^{G}\hat{\mathbf{T}}_{I_k}^{\kappa}\, {}^{I}\hat{\mathbf{T}}_{L_k}^{\kappa}\, {}^{L}\mathbf{p}_j - {}^{G}\mathbf{q}_j\right), \tag{3.4}$$

$$\mathbf{K} = \left(\mathbf{H}^T\mathbf{R}^{-1}\mathbf{H} + \mathbf{P}^{-1}\right)^{-1}\mathbf{H}^T\mathbf{R}^{-1}, \tag{3.5}$$

$$\hat{\mathbf{x}}_k^{\kappa+1} = \hat{\mathbf{x}}_k^\kappa \boxplus \left(-\mathbf{K}\mathbf{z}_k^\kappa - (\mathbf{I} - \mathbf{K}\mathbf{H})(\mathbf{J}^\kappa)^{-1}(\hat{\mathbf{x}}_k^\kappa \boxminus \hat{\mathbf{x}}_k)\right). \tag{3.6}$$

Algorithm 2 State Estimation with IKF

Input : Last output $\bar{\mathbf{x}}_{k-1}$ and $\bar{\mathbf{P}}_{k-1}$;
LiDAR raw points in current scan;
IMU inputs ($\mathbf{a}_m$, $\boldsymbol{\omega}_m$) during current scan.
Forward propagation to obtain state prediction $\hat{\mathbf{x}}_k$ and its covariance $\hat{\mathbf{P}}_k$ via Eq. (3.3);
Backward propagation to compensate motion [1];
$\kappa = -1$, $\hat{\mathbf{x}}_k^{\kappa=0} = \hat{\mathbf{x}}_k$;
repeat
$\kappa = \kappa + 1$;
Compute $\mathbf{J}^\kappa$ as in [2] and $\mathbf{P} = (\mathbf{J}^\kappa)^{-1}\hat{\mathbf{P}}_k(\mathbf{J}^\kappa)^{-T}$;
Compute residual $\mathbf{z}_j^\kappa$ and Jacobian $\mathbf{H}_j^\kappa$ as in [2];
Compute the state update $\hat{\mathbf{x}}_k^{\kappa+1}$ via (3.5 and 3.6);
until $\|\hat{\mathbf{x}}_k^{\kappa+1} \boxminus \hat{\mathbf{x}}_k^{\kappa}\| < \epsilon$;
$\bar{\mathbf{x}}_k = \hat{\mathbf{x}}_k^{\kappa+1}$, $\bar{\mathbf{P}}_k = (\mathbf{I} - \mathbf{KH})\mathbf{P}$;
Obtain the transformed LiDAR points $\{{}^G\bar{\mathbf{p}}_j\}$.
Output: Current optimal estimate $\bar{\mathbf{x}}_k$ and $\bar{\mathbf{P}}_k$;
The transformed LiDAR points $\{{}^G\bar{\mathbf{p}}_j\}$.

Sequential state estimation with LIO suffers from drift over long trajectories due to inherent noise in the robot motion, sensor measurements, data association problems, or dynamics in the environment [15]. Due to such a drift, when the robot revisits the same place, its belief may indicate a different location than it already knows. To overcome this problem, we proposed to integrate PGO with LIO so that such accumulated errors are corrected. PGO adds loop closure constraints to the robot poses when the same locations are revisited. To add loop closure constraints, a methodology to efficiently detect loop closure is required. In the next section, we discussed a robust loop closure detection methods and elaborated on one method we used in our research.

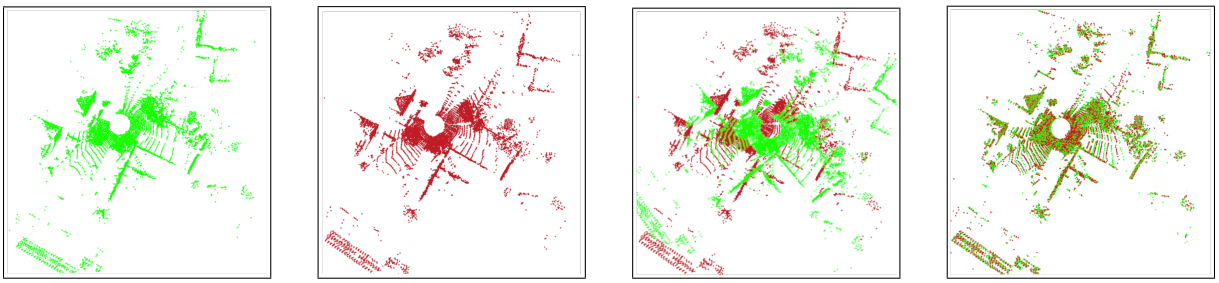
3.3 Loop Closure Detection

To maintain accurate pose estimate by mitigating global drift, loop closure via recognizing revisited places plays a vital role in SLAM with PGO. Therefore, a robust and efficient methodology for detecting loop closures is required to enforce a strong constraint in the graph. This enables us to have not only a better estimate of the current state and better model of the environment but also a better estimate of where the robot was in the past, since all the past poses will be updated as well.

Even though having many loop closure constraints in the graph improves our estimate, special attention should be paid not include false positive constraints by incorrect

association or perceptual aliasing in the graph because they will introduce a large error into the our estimation and it is difficult to remove them once they are added. It is better to miss a true loop closures than it is to add a false one. It is important to emphasize that loop closure relies on absolute measurements of the environment, typically obtained from sensors such as LiDAR, cameras, or other environmental sensing devices. Relative sensors, including the IMU or wheel encoders, are insufficient for performing loop closure. Therefore, using these relative sensors for detecting loops should be avoided.

3.3.1 Scan Context



(a) (b) (c) (d)

Figure 3.2: Loop detected with Scan Contex. a) current scan(source), b) target scan in the map, c) before alignment d) after alignment with ICP

As described in 2.7, Scan Context is a novel spatial descriptor and matching algorithm designed for outdoor place recognition using a single 3D scan. It encodes the entire point cloud from a 3D scan into a 2D matrix representation of size $N_r \times N_s$ (figure 3.3) where N_r denotes the number of rings and N_s is the number of sectors. These two variables parameters that can be tuned based on the environment. The radial resolution of the rings for LiDAR with maximum range set to L_{max} is $\frac{L_{max}}{N_r}$ and the angle of one sector is $\frac{2\pi}{N_s}$.

A bin is represented as a set of points that belong to *ith* ring and *jth* sector of the 2D partition where $i \in [1, N_r]$ and $j \in [1, N_s]$. For each bin, the maximum height (z value) of points falling into the bin is stored. This way, an intensity image also known as *Scan Context* of the point cloud in a single scan is generated as in Eq. (3.8) where ϕ is a function that assigns the maximum height of the points in the given bin represented by i and j.

Detecting loop closure requires measuring the similarity between scan contexts I_q and I_c of the query and candidate scan point clouds. This similarity score is the sum of the cosine distance between columns of same index in the scan contexts, c_j^q and c_j^c, divided by N_s for normalization Eq.(3.7). In our implementation, we used values between **0.1** to **0.13** for $d(I^q, I^c)$ to make very strict loop closure detection to avoid false positive loops and outliers. Using a smaller value for this parameter may lead to missing important loop closures. Conversely, using a larger value can introduce false loops, which increases map divergence and the size of the graph, ultimately degrading performance. Therefore, careful tuning is essential to a balance between avoiding missed loops and preventing the inclusion of false ones.

$$d(I^q, I^c) = \frac{1}{N_s} \sum_{j=1}^{N_s} \left(1 - \frac{c_j^q \cdot c_j^c}{\|c_j^q\| \, \|c_j^c\|} \right) \tag{3.7}$$

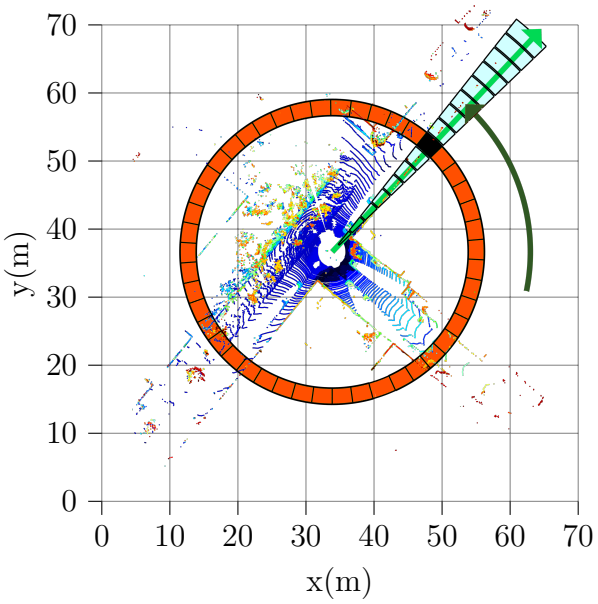


Figure 3.3: A 2D bin encoding of the 3D point cloud in a single scan

The column-wise similarity scoring of scan contexts is effective for dynamic objects, however columns in the scan context for the same place can be shifted due to a change in the view point of the LiDARR sensor. To alleviate this problem, distance between query scan context and all possible column-shifted scan contexts from the original candidate scan context is computed and the shift number n that gives the minimum distance is used as initial guess for alignment Eq.(3.9).

$$I = (a_{ij}) \in \mathbb{R}^{N_r \times N_s}, \quad a_{ij} = \phi(\mathcal{P}_{ij}) \tag{3.8}$$

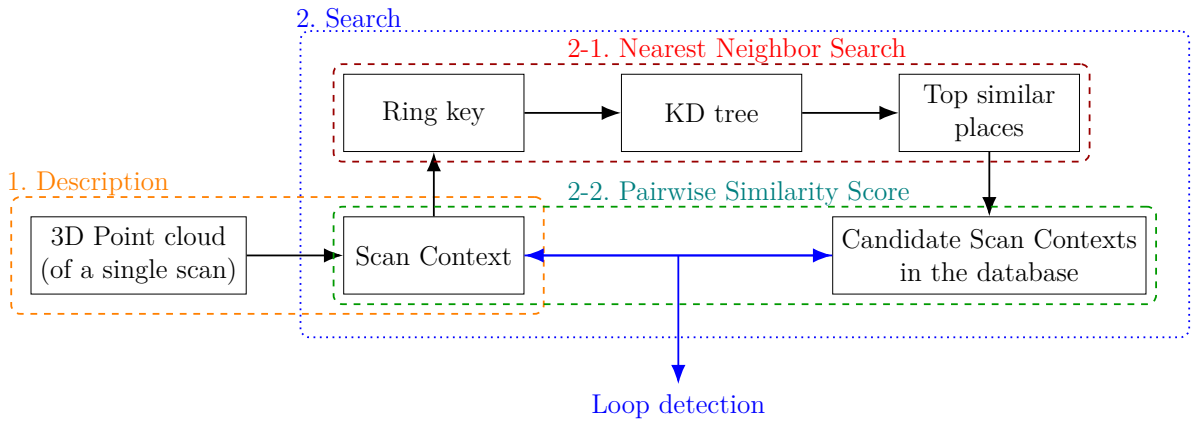


Figure 3.4: A point cloud from a single 3D scan is encoded into a Scan Context. An N_r dimensional ring key is then extracted and used to build a KD-tree for retrieving nearest candidates. These candidates are compared with the query, and the closest one that meets the acceptance threshold is selected as a loop closure[16].

$$\begin{aligned} D(I^q, I^c) &= \min_{n \in [N_s]} d(I^q, I^c_n), \\ n^* &= \arg\min_{n \in [N_s]} d(I^q, I^c_n) \end{aligned} \tag{3.9}$$

Algorithm 3 Loop Detection with Scan Context

Input: Current descriptor I_q, current key k_q
Output: Loop ID n^* and yaw difference $\Delta\theta$
if number of stored descriptors $< N_{\text{exclude}} + 1$ **then**
    return $(-1, 0.0)$ [avoid loop with recent scans]
end if
if tree update required (every T iterations to save computation) **then**
    Exclude recent N_{exclude} keys from search set
    Build KD-tree on historical keys
end if
Use KD-tree to retrieve top-K candidates $\{k_n\}$ closest to k_q
for each candidate $n \in \{k_n\}$ **do**
    Compute pairwise scan context distance $d(I_q, I_n)$ with optimal $\Delta\theta$
end for
Select best match using 3.9
if $D^* < \tau_{\text{SC}}$ **then**
    return $(n^*, \Delta\theta)$
else
    return $(-1, 0.0)$
end if

The number of top candidate scan contexts K in algorithm 3 is determined by the user and the candidate with the minimum distance to the query descriptor satisfying the

acceptable threshold, τ_{SC}, is chosen as revisited place. Figure 3.2 shows the query(green) and candidate(red) scan satisfying the acceptable similarity threshold aligned with the initial guess $\Delta\theta$, and ICP.

3.4 ICP for computing loop closure constraint

ICP [20] is a widely used algorithm for rigid registration of 3D point clouds. It operates by iteratively refining the alignment between a source and a target point cloud by minimizing the sum of squared distances between corresponding points. In each iteration, it first identifies the closest points between the two clouds, estimates the best-fit rigid transformation (rotation and translation), applies this transformation to the source, and repeats the process until convergence. The optimal rotation R is computed using singular value decomposition (SVD) [21] of the cross covariance matrix of the $\mathbf{H}$ shown in Eq. (3.11) of the corresponding points where x_0 and y_0 are the weighted mean of point clouds X, and Y respectively. The basic alignment problem of point sets X and Y is shown in Eq. (3.10) where x_i and y_j are corresponding points that belong to 3D point clouds X and Y respectively.

$$(\mathbf{R}^*, \mathbf{t}^*) = \arg\min_{\mathbf{R},\mathbf{t}} \sum_{(i,j)\in\mathcal{C}} \|\mathbf{y}_i - \mathbf{R}\mathbf{x}_j - \mathbf{t}\|^2 \tag{3.10}$$

$$\mathbf{H} = \sum (y_i - y_0)(x_j - x_0)^\top, \quad \mathbf{U}\mathbf{D}\mathbf{V}^\top = \text{SVD}(\mathbf{H}), \quad \mathbf{R}^* = \mathbf{U}\mathbf{V}^\top, \quad \mathbf{t}^* = \mathbf{y}_0 - \mathbf{R}\mathbf{x}_0 \tag{3.11}$$

Table 3.1: Parameters for ICP (Point-to-Point)

Symbol	Description	Value
$d_{\max}$	Maximum correspondence distance	150 m
N_{iter}	Maximum number of iterations	50
ϵ_{trans}	Minimum transformation epsilon (convergence)	1×10^{-6}
ϵ_{fit}	Euclidean fitness epsilon (convergence)	1×10^{-6}
N_{RANSAC}	RANSAC iterations for outlier rejection	0
τ_{RANSAC}	RANSAC outlier rejection threshold	0.05 m

Point Cloud Libarary (PCL), among others, provides implementation for several variants of ICP algorithm. We used the point-to-point implementation of ICP from the PCL library. After a pair of scans corresponding to the detected loop are found, we run the PCL's ICP with the parameters shown in table 3.1. It is important to tune these parameters properly for optimal alignment of the points clouds in the detected scans that correspond to loops.

However, in our method, since the loop closure detection module robustly detects loops with $\Delta\theta$ as initial guess, these parameters shown in 3.1 do not require excessive tuning. For short-range loop closure and indoor environments, tuning these parameters is required accordingly. The transformation ($\mathbf{R}^*$ and $\mathbf{t}^*$) that aligns the two scans corresponding to the loop is added as *loop constraint* to the factor graph.

3.5 Optimization with Pose Graph

By leveraging odometry constraints derived from LIO and loop closure constraints obtained through ICP alignment, we construct a pose graph that is subsequently optimized (see algorithm 1) to minimize a global error function shown in Eq. (3.13). In this framework, odometry constraints encode the relative motion estimates (transformation) between temporally adjacent poses, while loop closure constraints introduce corrections by identifying correspondences between non-consecutive poses, thereby mitigating accumulated drift and enhancing global consistency. Minimization of the global error function constructs configuration of the robot poses that best satisfies the constraints [22].

$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j). \tag{3.12}$$

$$\mathbf{F}(\mathbf{x}) = \sum_{\langle i,j \rangle \in \mathcal{C}} \underbrace{\mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathbf{F}_{ij}} \tag{3.13}$$

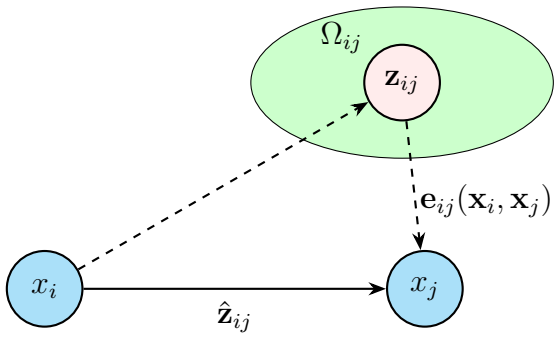


Figure 3.5: Error between actual measurement $\mathbf{z}_{ij}$ and expected measurement $\hat{\mathbf{z}}_{ij}$

For two nodes i and j in the pose graph, the function that computes the difference between the actual measurement $\mathbf{z}_{ij}$ and expected measurement $\hat{\mathbf{z}}_{ij}$ is denoted as in Eq. (3.12) [22]. For a set C of pairs of indices which have a constraint (observation) $\mathbf{z}_{ij}$ with information

matrix $\mathbf{\Omega}_{ij}$, maximizing the PDF Eq. (2.19) corresponds to finding the configuration of nodes $\mathbf{x}^*$ that minimizes the negative log likelihood $\mathbf{F(x)}$ of all observations as in Eq. (3.14).

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} \mathbf{F}(\mathbf{x}). \tag{3.14}$$

The solution to Eq. (3.14) can be found utilizing popular optimization methods such as Gauss-Newton [23] or the Levenberg-Marquardt (LM) [24] algorithms.

3.6 Global refinement using HBA

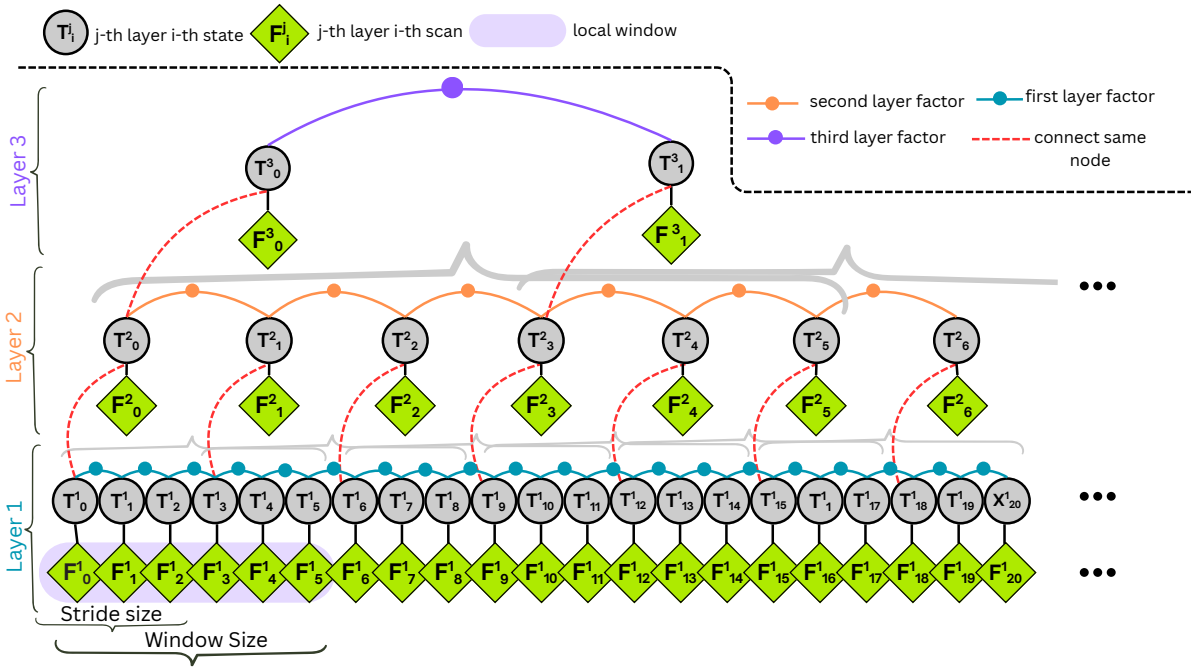


Figure 3.6: Pyramid structure of the HBA with layer number l=3, stride size s=3 and window size w=6.

Minimization of the global error function Eq.(3.13) generates best estimate of the robot poses that best fit the measurements under Gaussian assumption of the relative pose estimation. This minimization corrects the drift in the LiDAR odometry and inconsistency in the map. However, due to loop closure inaccuracy and optimization of only relative pose, there would still be small misalignments in the map after PGO. As discussed in section 2.5.1, LBA works to minimize the overall point-to-plane distance to optimize the mapping consistency. However this optimization process requires $\mathbf{O}(N^3)$ time to solve where N is the number of LiDAR poses involved in the computation. To address this issue, Xiyuan Liu

et al. [17] proposed HBA method that globally optimizes the map consistency maintaining the efficiency of time consumption.

The architecture of HBA, as shown in figure 3.6, constructs pyramid structure of the frame poses. The refinement process involves two key steps. A bottom-up approach that performs BA within local window hierarchically from bottom layer(local BA) to the top layer(global BA). Since the number frame poses in each local window is small, this approach of employing local BA improves the time complexity by leveraging parallel computation for each local BA. However, this bottom-up process does not take into account the features that are common across different local windows and this causes inaccuracy. To mitigate this issue, a top-down process that constructs PGO from top to bottom layers and distributes the errors is used.

3.6.1 Bottom-up Process

For j-th LiDAR frame $\mathbb{F}_j^i$ in the i-th layer and its pose $\mathbf{T}_j^i = (\mathbf{R}_j^i, \mathbf{t}_j^i)$, with $\mathbf{R}_j^i \in SO(3)$ and $\mathbf{t}_j^i \in \mathbb{R}^3$, its points are represented in the local LiDAR frame and $\mathbf{T}_j^i$ is in global frame. A local BA is performed on the frames in each local window of size w and stride s in the bottom-up process given that the initial pose trajectory is provided. This local BA optimizes the relative poses between the first frame and the other frames within a local window. After optimization, a new key frame for the next layer Eq. (3.15) is created for each window by aggregating all the frames inside it. The pose of this key frame, Eq. (3.17), is computed by sequentially multiplying the optimized poses of the individual LiDAR frames in the window. This process is repeated from the bottom layer to top layer of the pyramid structure. This local BA forms relative constraints(factors) between the frames within the window. For keyframes that are appear in different windows, multiple relative constraints from different windows will constructed.

$$\mathbf{F}_j^{i+1} \triangleq \bigcup_{k=0}^{w-1} \left(\mathbf{T}_{s,j,s,j+k}^{i*} \cdot \mathbf{F}_{s,j+k}^{i} \right) \tag{3.15}$$

The total time consumption of HBA depends on the number of layers l, total number of LiDAR frames N, window size w, stride number s, and the number of threads n used for parallel processing. The optimal number of layers l^* of the pyramid is obtained as in Eq.(3.16).

$$l^* = \left\lfloor \frac{1}{2} \log_s \left(\frac{3N^2(s^3 - s)n}{w^3} \right) \right\rfloor \tag{3.16}$$

$$\mathbf{T}_j^{i+1} = \mathbf{T}_{s,j}^{i*} = \prod_{k=1}^{j} \mathbf{T}_{s,k-s,s,k}^{i*} \tag{3.17}$$

3.6.2 Top-Down Process

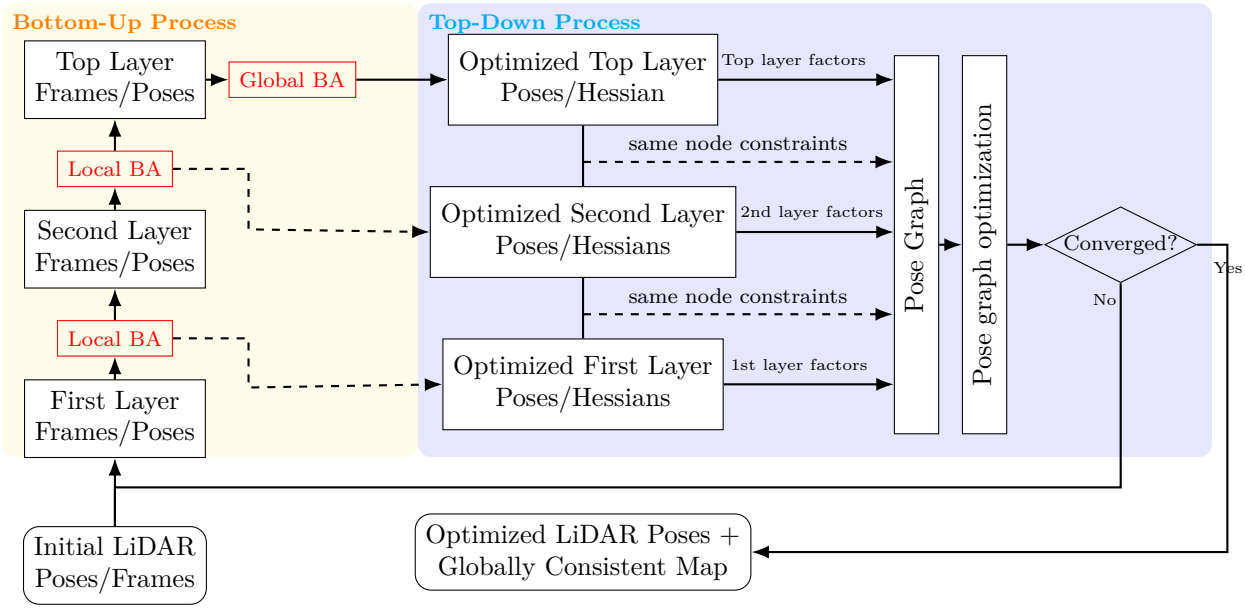


Figure 3.7: HBA optimization: Bottom-Up and Top-Down Process

In the top-down process,see figure 3.7, pose graph is constructed using the frame poses as variables and relative pose between adjacent frames as factors in each layer of the pyramid. Finally PGO is used to solve the pose graph by LM using GTSAM [25].

Chapter 4

Experiments and Results

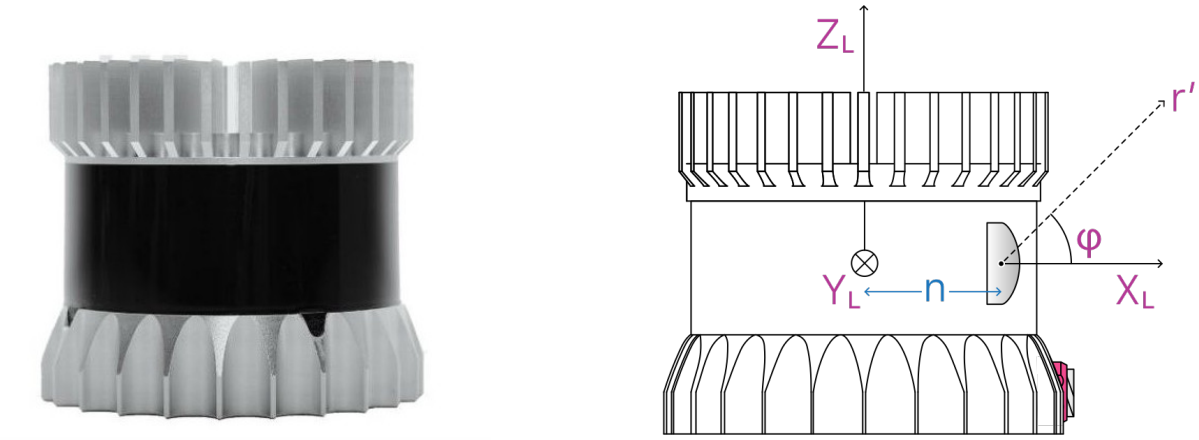
4.1 Introduction

This chapter presents the evaluation of the proposed mapping system through a series of experiments conducted on both custom and public datasets. The custom dataset includes trajectories of varying lengths (long and short) captured in diverse settings, containing both indoor and outdoor environments. The aim is to assess the system's performance in terms of trajectory accuracy, loop closure effectiveness, map consistency, and computational efficiency.

4.2 Experimental Setup

4.2.1 Hardware and Platform

The custom datasets were collected using the Hunter 2.0 mobile robot from AgileX Robotics, equipped with an Ouster OS1-128 LiDAR. The sensor provides 128 scan lines, a 360° horizontal field of view, and operates at 10 Hz. It also includes a built-in IMU, which was used for LIO odometry estimation. The extrinsic calibration between the ouster LiDAR and the IMU can be computed online with FAST-LIO2, however in our experiments we used identity for rotation with no translation. All data were recorded using ROS 2 bag files for both real-time and offline processing. For running the mapping system, both in real-time and during offline refinement, a separate computer was used. It featured an AMD Ryzen 7 CPU, NVIDIA RTX 3060 GPU, and 32 GB RAM, running Ubuntu 22.04 with ROS 2 Humble. All mapping modules were implemented in C++ to ensure real-time performance and efficient offline processing.



(a) (b)

Figure 4.1: a) Ouster OS1-128 LiDAR with built-in IMU. b) Sensor coordinate frame [26]

4.2.2 Software Modules

The entire mapping pipeline contains modular components implemented in C++ within ROS2 Humble framework. The software dependencies of each components is summerized in table 4.1. The version numbers we used in our experiments are shown in bold.

Table 4.1: Software modules and dependencies

Module / Deps	FAST-LIO2	PGO	HBA
C++	17		
Eigen	>=3.3.4(**3.4.0**)	**3.4.0**	>=3.3.7(**3.4.0**)
GTSAM	None	>=4.1.1(**4.2.0**)	
PCL	>=1.8(**1.12.1**)	**1.12.1**	>=1.10.0(**1.12.1**)
Ubuntu	22		
ROS	Humble		

4.2.3 Dataset

put desciption of the dataset and the sensors used in each and parameters things like clearly.

To evaluate the performance of our mapping module, we performed experiments with various open source dataset and our own dataset collected around Saxion building. The experiments performed with the open source dataset were performed to evaluate the robustness of our mapping module to different variety of sensors types, seasonal and environmental changes. open source dataset contains LiDAR data, IMU data, gps data,

ZED camera data. the LiDAR type is Ouster with 128 scan lines and 360deg FoV. The rate of the IMU, and LiDAR .....

Table 4.2: Dataset used for evaluating performance of our mapping module

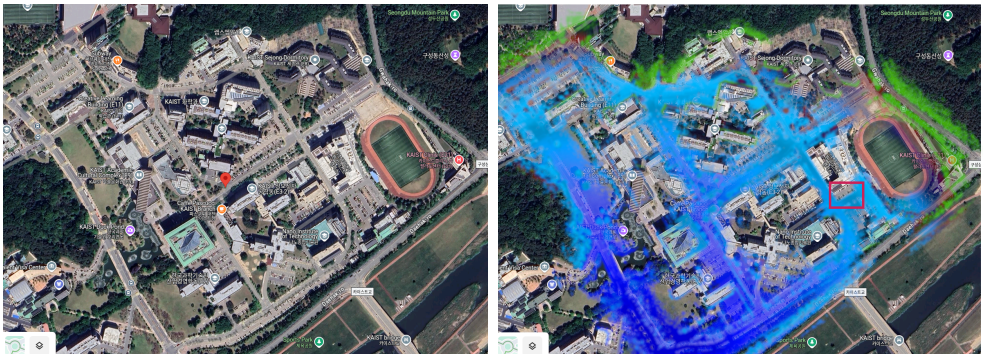
Dataset	LiDAR	Envt.	Ground Truth	Size	Distance(Km)	Duration(s)
Saxion-01	OS-128	Outdoor	✗	15GB		???
Saxion-02	OS-128	Outdoor	✗	15GB		???
Saxion-03	OS-128	Outdoor	✗	15GB		???
Saxion-11	OS-128	Indoor	✗	15GB		???
KITTI05	V-64??	Outdoor	✓	18GB		???
KAIST03	OS1-64	Outdoor	✓	18GB		???

In order for the SLAM to work correctly for each dataset, an extrinsic calibration between the IMU and LiDAR sensors is required. Table 4.2 the calibration matrix for each dataset used in the experiments.

4.2.4 Analysis of Accuracy

Figure **??** shows the result of the map and trajectory before and after applying optimization to the output of the LiDAR odometry based poses. It can be clearly seen in 4.2b that the map generated by odometry is not accurate and it has duplication in most part of the map specially on the top right corner side.

As initial step, we tested on the Mulran KAIST02 sequence using the odometry from FAST-LIO2 and generated map both from the odometry and pose-graph optimization.



(a) KAIST google map (b) Map overlayed to google map

Figure 4.2: Odometry showing direction of driving(not fully optimized because both optimization and publishing the odometry run in separate threads so the published odometry is not final optimized result)

The map shown in **??** is generated after the FAST-LIO2 poses are optimized by ISAM2 after adding loop closure constraints to the factor graph. Since this map is generated from the optimized poses, the duplications are now removed and is more consistent than the previous map. The drift accumulated by FAST-LIO2 odometry can be seen in figure **??** and figure **??** shows the trajectory after the drift is corrected with ISAM2.

To further analyze the accuracy of the optimized map, we overlayed the generated map onto the google map, and it can be seen that some small inconsistencies are visible because unlike LiDAR bundle adjustment methods [**bibid**], pose graph optimization does not optimize point cloud consistency directly[27]. Instead, it focuses solely in optimizing relative pose errors. Hence the divergence is not completely eliminated in the map.

Table 4.3: Metric comparison of our SLAM with state-off-the-art for indoor custom indoor dataset at Saxion

Method	avg runtime	avg freq.	Num loop closures
KISS-SLAM	30ms	34.00 HZ	77
MOLA SLAM	29.72ms	33.65HZ	-
PROPOSED			

Table 4.3 shows the comparison of our method with most recent SLAM methods, and it is shown that KISS-SLAM results in many false positive loop closures resulting inaccuracy in the generated map. The average runtime of both SLAM methods is comparably real-time.

4.3 Evaluation metrics

The proposed mapping pipeline was evaluated using a comprehensive set of quantitative and qualitative metrics to ensure a robust performance assessment. Specifically Absolute Pose Error(APE) was employed to evaluate the global consistency of the estimated trajectory by comparing it against the ground truth data. To measure local accuracy and drift between consecutive poses, Relative Pose Error (RPE) was employed. Map consistency was assessed qualitatively through visual inspection of the generated maps. Finally, Runtime Performance was measured by recording the average processing time per LiDAR scan in the online module. These evaluation criteria collectively provide a thorough insight into the accuracy, robustness, and efficiency of the developed system.

4.4 Results

4.4.1 Trajectory accuracy

4.4.2 Map quality and consistency

4.4.3 Runtime performance

Chapter 5

Conclusion

5.1 Future work

Use deep learning methods for loop closure detection to improve efficiency, and improving time efficiency of the overall system. Geo-referencing the SLAM map so that it is anchored to absolute global position systems.

Bibliography

[1] Wen Xu et al. “FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by Tightly-coupled Iterated Kalman Filter”. In: *IEEE Robotics and Automation Letters* 6 (2021), pp. 7270–7276.

[2] Wei Xu et al. “Fast-lio2: Fast direct lidar-inertial odometry”. In: *IEEE Transactions on Robotics* 38.4 (2022), pp. 2053–2073.

[3] Tixiao Shan et al. “Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping”. In: *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE. 2020, pp. 5135–5142.

[4] Joan Sola, Jeremie Deray, and Dinesh Atchuthan. “A micro lie theory for state estimation in robotics”. In: *arXiv preprint arXiv:1812.01537* (2018).

[5] Davide Scaramuzza and Friedrich Fraundorfer. “Visual odometry [tutorial]”. In: *IEEE robotics & automation magazine* 18.4 (2011), pp. 80–92.

[6] Jiarong Lin and Fu Zhang. “Loam livox: A fast, robust, high-precision LiDAR odometry and mapping package for LiDARs of small FoV”. In: *2020 IEEE international conference on robotics and automation (ICRA)*. IEEE. 2020, pp. 3126–3131.

[7] Zheng Liu and Fu Zhang. “Balm: Bundle adjustment for lidar mapping”. In: *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 3184–3191.

[8] Xiyuan Liu et al. “Large-Scale LiDAR Consistent Mapping Using Hierarchical LiDAR Bundle Adjustment”. In: *IEEE Robotics and Automation Letters* 8.3 (2023), pp. 1523–1530. DOI: 10.1109/LRA.2023.3238902.

[9] Guoquan P Huang, Anastasios I Mourikis, and Stergios I Roumeliotis. “Analysis and improvement of the consistency of extended Kalman filter based SLAM”. In: *2008 IEEE International Conference on Robotics and Automation*. IEEE. 2008, pp. 473–479.

[10] Sebastian Thrun. "Probabilistic robotics". In: *Communications of the ACM* 45.3 (2002), pp. 52–57.

[11] Frank Dellaert, Michael Kaess, et al. "Factor graphs for robot perception". In: *Foundations and Trends® in Robotics* 6.1-2 (2017), pp. 1–139.

[12] Jose Luis Blanco-Claraco. "A flexible framework for accurate LiDAR odometry, map manipulation, and localization". In: *The International Journal of Robotics Research* 0.0 (2025), p. 02783649251316881. DOI: 10.1177/02783649251316881. eprint: https://doi.org/10.1177/02783649251316881. URL: https://doi.org/10.1177/02783649251316881.

[13] Tiziano Guadagnino et al. "KISS-SLAM: A Simple, Robust, and Accurate 3D LiDAR SLAM System With Enhanced Generalization Capabilities". In: *arXiv preprint arXiv:2503.12660* (2025).

[14] Ji Zhang, Sanjiv Singh, et al. "LOAM: Lidar odometry and mapping in real-time." In: *Robotics: Science and systems*. Vol. 2. Berkeley, CA. 2014, pp. 1–9.

[15] S. Gupta et al. "Effectively Detecting Loop Closures using Point Cloud Density Maps". In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2024.

[16] Giseop Kim and Ayoung Kim. "Scan context: Egocentric spatial descriptor for place recognition within 3d point cloud map". In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2018, pp. 4802–4809.

[17] Marian Himstedt et al. "Large scale place recognition in 2D LIDAR scans using geometrical landmark relations". In: *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2014, pp. 5030–5035.

[18] Naveed Muhammad and Simon Lacroix. "Loop closure detection using small-sized signatures from 3D LIDAR data". In: *2011 IEEE International Symposium on Safety, Security, and Rescue Robotics*. IEEE. 2011, pp. 333–338.

[19] Walter Wohlkinger and Markus Vincze. "Ensemble of shape functions for 3D object classification". In: *2011 IEEE international conference on robotics and biomimetics*. IEEE. 2011, pp. 2987–2992.

[20] K. S. Arun, T. S. Huang, and S. D. Blostein. "Least-Squares Fitting of Two 3-D Point Sets". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-9.5 (1987), pp. 698–700. DOI: 10.1109/TPAMI.1987.4767965.

[21] Gene H Golub and Christian Reinsch. "Singular value decomposition and least squares solutions". In: *Handbook for automatic computation: volume II: linear algebra*. Springer, 1971, pp. 134–151.

[22] Giorgio Grisetti et al. "A tutorial on graph-based SLAM". In: *IEEE Intelligent Transportation Systems Magazine* 2.4 (2010), pp. 31–43.

[23] Yong Wang. "Gauss–newton method". In: *Wiley Interdisciplinary Reviews: Computational Statistics* 4.4 (2012), pp. 415–420.

[24] Sam Roweis. "Levenberg-marquardt optimization". In: *Notes, University Of Toronto* 52 (1996), p. 1027.

[25] Frank Dellaert and GTSAM Contributors. *borglab/gtsam*. Version 4.2a8. May 2022. DOI: 10.5281/zenodo.5794541. URL: https://github.com/borglab/gtsam).

[26] inc. Ouster. *Ouster sensors docs*. https://static.ouster.dev/sensor-docs/image_route1/image_route3/sensor_data/sensor-data.html. Accessed: 2025-05-02. 2024.

[27] Yue Pan et al. "MULLS: Versatile LiDAR SLAM via Multi-metric Linear Least Square". In: *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE. 2021.

List of Figures